CS201 Solved Practice Programs

28 logic-building programs in C++, from basic to advanced

How to use this book

1. Read the one-line idea, then close the page and try to write the program yourself.
2. Run your version and compare its output with the output shown here.
3. Study the solution and retype it from memory the next day.
4. Every program was compiled and run, and the output shown is the real result.

Compile command: `g++ program.cpp -o program`

#	Program	Handout
Level 1: Loops and Conditions		
1	Prime number check	L5-L8
2	Fibonacci series and factorial	L6-L7
3	Reverse a number and check palindrome	L6
4	Count digits and sum of digits	L6
5	Armstrong number	L6-L7
6	Star patterns: pyramid, number triangle, diamond	L6-L7
Level 2: Arrays		
7	Largest, smallest and second largest	L11-L12
8	Reverse an array in place	L11-L12
9	Linear search and binary search	L11-L12
10	Bubble sort and selection sort	L12
11	Remove duplicates from an array	L11-L12
12	Find the missing number from 1 to n	L11-L13
Level 3: 2D Arrays and Matrices		
13	Print a 5x5 matrix in reverse column order	L12-L13 (handout exercise)
14	Transpose of a matrix	L12-L13 (handout exercise)
15	Check for an identity matrix	L12-L13 (handout exercise)
16	Add and multiply two matrices	L12-L13
17	Sum of both diagonals	L12-L13
Level 4: Strings		
18	Write your own strlen, strcpy, strcmp, strcat	L12, L17 (handout exercise)
19	Reverse a string and check palindrome	L12
20	Count vowels, consonants and words	L12
21	Check whether two words are anagrams	L12
22	Print the ASCII table	L17 (handout exercise)
Level 5: Pointers		
23	Swap two numbers using pointers	L14

24	Traverse an array using pointer arithmetic	L15
25	Bubble sort using pointers	L14 (handout sample)
26	Fill, print and add an array through pointers	L14-L15
Level 6: Structures and Files		
27	Student records: array of structures, find the topper	L20
28	Write text to a file, then read it back and count lines	L18

Level 1: Loops and Conditions

Program 1: Prime number check

Handout reference: L5-L8

A number is prime if nothing from 2 up to its square root divides it evenly.

SOLUTION

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter a number:" << endl;
    cin >> n;

    bool prime = (n >= 2);                  // 0 and 1 are not prime
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {                   // found a divisor
            prime = false;
            break;
        }
    }
    cout << n << (prime ? " is prime" : " is not prime") << endl;
    return 0;
}
```

SAMPLE INPUT

```
29
```

OUTPUT

```
Enter a number:
29 is prime
```

Program 2: Fibonacci series and factorial

Handout reference: L6-L7

Fibonacci: each term is the sum of the previous two. Factorial: multiply 1 to n.

SOLUTION

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter n:" << endl;
    cin >> n;

    long long a = 0, b = 1;
    cout << "First " << n << " Fibonacci terms: ";
    for (int i = 0; i < n; i++) {
        cout << a << " ";
        long long next = a + b;            // slide the window forward
        a = b;
        b = next;
    }

    long long fact = 1;
    for (int i = 2; i <= n; i++) fact *= i;
    cout << "\nFactorial of " << n << " = " << fact << endl;
    return 0;
}
```

SAMPLE INPUT

```
8
```

OUTPUT

```
Enter n:
First 8 Fibonacci terms: 0 1 1 2 3 5 8 13
Factorial of 8 = 40320
```

Program 3: Reverse a number and check palindrome

Handout reference: L6

Peel off the last digit with % 10, build the reverse, then compare with the original.

SOLUTION

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter a number:" << endl;
    cin >> n;

    int original = n, reversed = 0;
    while (n > 0) {
        reversed = reversed * 10 + n % 10;  // append the last digit
        n /= 10;                            // drop the last digit
    }
    cout << "Reverse = " << reversed << endl;
    if (original == reversed)
        cout << original << " is a palindrome" << endl;
    else
        cout << original << " is not a palindrome" << endl;
    return 0;
}
```

SAMPLE INPUT

```
12321
```

OUTPUT

```
Enter a number:
Reverse = 12321
12321 is a palindrome
```

Program 4: Count digits and sum of digits

Handout reference: L6

The same digit-peeling loop, now counting digits and adding them up.

SOLUTION

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter a number:" << endl;
    cin >> n;

    int count = 0, sum = 0, temp = n;
    if (temp == 0) count = 1;              // the number 0 has one digit
    while (temp > 0) {
        sum += temp % 10;
        count++;
        temp /= 10;
    }
    cout << "Digits = " << count << endl;
    cout << "Sum of digits = " << sum << endl;
    return 0;
}
```

SAMPLE INPUT

```
4521
```

OUTPUT

```
Enter a number:
Digits = 4
Sum of digits = 12
```

Program 5: Armstrong number

Handout reference: L6-L7

A number equals the sum of its digits, each raised to the power of the digit count.

SOLUTION

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter a number:" << endl;
    cin >> n;

    int digits = 0;
    for (int t = n; t > 0; t /= 10) digits++;

    int sum = 0;
    for (int t = n; t > 0; t /= 10) {
        int d = t % 10, power = 1;
        for (int i = 0; i < digits; i++) power *= d;   // d raised to 'digits'
        sum += power;
    }
    if (sum == n) cout << n << " is an Armstrong number" << endl;
    else          cout << n << " is not an Armstrong number" << endl;
    return 0;
}
```

SAMPLE INPUT

```
153
```

OUTPUT

```
Enter a number:
153 is an Armstrong number
```

Program 6: Star patterns: pyramid, number triangle, diamond

Handout reference: L6-L7

Nested loops: the outer loop picks the row, the inner loops print spaces and symbols.

SOLUTION

```
#include <iostream>
using namespace std;

void row(int spaces, int stars) {
    for (int i = 0; i < spaces; i++) cout << ' ';
    for (int i = 0; i < stars; i++)  cout << '*';
    cout << endl;
}

int main() {
    int n = 4;

    cout << "Pyramid:" << endl;
    for (int i = 1; i <= n; i++) row(n - i, 2 * i - 1);

    cout << "\nNumber triangle:" << endl;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) cout << j << " ";
        cout << endl;
    }

    cout << "\nDiamond:" << endl;
    for (int i = 1; i <= n; i++)     row(n - i, 2 * i - 1);   // upper half
    for (int i = n - 1; i >= 1; i--) row(n - i, 2 * i - 1);   // lower half
    return 0;
}
```

OUTPUT

```
Pyramid:
   *
  ***
 *****
*******

Number triangle:
1
1 2
1 2 3
1 2 3 4

Diamond:
   *
  ***
 *****
*******
 *****
  ***
   *
```

Level 2: Arrays

Program 7: Largest, smallest and second largest

Handout reference: L11-L12

One pass finds the largest and smallest. A second pass finds the best value below the largest.

SOLUTION

```
#include <iostream>
#include <climits>
using namespace std;

int main() {
    const int N = 6;
    int a[N];
    cout << "Enter 6 numbers:" << endl;
    for (int i = 0; i < N; i++) cin >> a[i];

    int largest = a[0], smallest = a[0];
    for (int i = 1; i < N; i++) {
        if (a[i] > largest)  largest = a[i];
        if (a[i] < smallest) smallest = a[i];
    }

    int second = INT_MIN;                  // smaller than any possible value
    for (int i = 0; i < N; i++)
        if (a[i] != largest && a[i] > second) second = a[i];

    cout << "Largest = " << largest << endl;
    cout << "Second largest = " << second << endl;
    cout << "Smallest = " << smallest << endl;
    return 0;
}
```

SAMPLE INPUT

```
45 12 89 7 63 33
```

OUTPUT

```
Enter 6 numbers:
Largest = 89
Second largest = 63
Smallest = 7
```

Program 8: Reverse an array in place

Handout reference: L11-L12

Two indices walk toward each other from both ends, swapping as they go.

SOLUTION

```
#include <iostream>
using namespace std;

int main() {
    const int N = 6;
    int a[N];
    cout << "Enter 6 numbers:" << endl;
    for (int i = 0; i < N; i++) cin >> a[i];

    int i = 0, j = N - 1;
    while (i < j) {
        int temp = a[i];
        a[i] = a[j];
        a[j] = temp;
        i++;
        j--;
    }

    cout << "Reversed array: ";
    for (int k = 0; k < N; k++) cout << a[k] << " ";
    cout << endl;
    return 0;
}
```

SAMPLE INPUT

```
1 2 3 4 5 6
```

OUTPUT

```
Enter 6 numbers:
Reversed array: 6 5 4 3 2 1
```

Program 9: Linear search and binary search

Handout reference: L11-L12

Linear search checks every element. Binary search halves a SORTED array each step.

SOLUTION

```
#include <iostream>
using namespace std;

int linearSearch(int a[], int n, int key) {
    for (int i = 0; i < n; i++)
        if (a[i] == key) return i;
    return -1;                              // not found
}

int binarySearch(int a[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (a[mid] == key)     return mid;
        else if (a[mid] < key) low = mid + 1;    // search right half
        else                   high = mid - 1;   // search left half
    }
    return -1;
}

int main() {
    int a[8] = {3, 8, 15, 23, 42, 57, 64, 91};   // already sorted
    int key;
    cout << "Enter the number to find:" << endl;
    cin >> key;

    cout << "Linear search index = " << linearSearch(a, 8, key) << endl;
    cout << "Binary search index = " << binarySearch(a, 8, key) << endl;
    return 0;
}
```

SAMPLE INPUT

```
42
```

OUTPUT

```
Enter the number to find:
Linear search index = 4
Binary search index = 4
```

Program 10: Bubble sort and selection sort

Handout reference: L12

Bubble sort swaps neighbours. Selection sort places the smallest remaining value each pass.

SOLUTION

```
#include <iostream>
using namespace std;

void printArray(int a[], int n) {
    for (int i = 0; i < n; i++) cout << a[i] << " ";
    cout << endl;
}

void bubbleSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
            }
}

void selectionSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[min]) min = j;    // remember the smallest position
        int t = a[i]; a[i] = a[min]; a[min] = t;
    }
}

int main() {
    int x[6] = {64, 25, 12, 22, 11, 90};
    int y[6] = {64, 25, 12, 22, 11, 90};

    bubbleSort(x, 6);
    selectionSort(y, 6);
    cout << "Bubble sort:    "; printArray(x, 6);
    cout << "Selection sort: "; printArray(y, 6);
    return 0;
}
```

OUTPUT

```
Bubble sort:    11 12 22 25 64 90
Selection sort: 11 12 22 25 64 90
```

Program 11: Remove duplicates from an array

Handout reference: L11-L12

Copy each element to a result array only if it has not been copied already.

SOLUTION

```
#include <iostream>
using namespace std;

int main() {
    int a[8] = {4, 2, 4, 7, 2, 9, 7, 1};
    int result[8];
    int count = 0;

    for (int i = 0; i < 8; i++) {
        bool seen = false;
        for (int j = 0; j < count; j++)
            if (result[j] == a[i]) { seen = true; break; }
        if (!seen) result[count++] = a[i];
    }

    cout << "Without duplicates: ";
    for (int i = 0; i < count; i++) cout << result[i] << " ";
    cout << endl;
    return 0;
}
```

OUTPUT

```
Without duplicates: 4 2 7 9 1
```

Program 12: Find the missing number from 1 to n

Handout reference: L11-L13

The sum of 1..n is n(n+1)/2. The missing number is that total minus the sum you entered.

SOLUTION

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter n (numbers are 1 to n, one is missing):" << endl;
    cin >> n;

    cout << "Enter the " << n - 1 << " numbers:" << endl;
    int sum = 0;
    for (int i = 0; i < n - 1; i++) {
        int x;
        cin >> x;
        sum += x;
    }

    int expected = n * (n + 1) / 2;
    cout << "Missing number = " << expected - sum << endl;
    return 0;
}
```

SAMPLE INPUT

```
7
1 2 4 5 7 3
```

OUTPUT

```
Enter n (numbers are 1 to n, one is missing):
Enter the 6 numbers:
Missing number = 6
```

Level 3: 2D Arrays and Matrices

Program 13: Print a 5x5 matrix in reverse column order

Handout reference: L12-L13 (handout exercise)

Rows stay the same. Only the inner loop runs backward, from the last column to the first.

SOLUTION

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    const int N = 5;
    int m[N][N];
    cout << "Enter 25 numbers:" << endl;
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            cin >> m[i][j];

    cout << "Reverse column order:" << endl;
    for (int i = 0; i < N; i++) {
        for (int j = N - 1; j >= 0; j--)   // columns counted backward
            cout << setw(4) << m[i][j];
        cout << endl;
    }
    return 0;
}
```

SAMPLE INPUT

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
```

OUTPUT

```
Enter 25 numbers:
Reverse column order:
   5   4   3   2   1
  10   9   8   7   6
  15  14  13  12  11
  20  19  18  17  16
  25  24  23  22  21
```

Program 14: Transpose of a matrix

Handout reference: L12-L13 (handout exercise)

Transpose swaps rows and columns: element [i][j] moves to [j][i].

SOLUTION

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    const int N = 3;
    int m[N][N], t[N][N];
    cout << "Enter 9 numbers:" << endl;
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            cin >> m[i][j];

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            t[j][i] = m[i][j];             // rows become columns

    cout << "Transpose:" << endl;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) cout << setw(4) << t[i][j];
        cout << endl;
    }
    return 0;
}
```

SAMPLE INPUT

```
1 2 3 4 5 6 7 8 9
```

OUTPUT

```
Enter 9 numbers:
Transpose:
   1   4   7
   2   5   8
   3   6   9
```

Program 15: Check for an identity matrix

Handout reference: L12-L13 (handout exercise)

Diagonal elements must all be 1 and every other element must be 0.

SOLUTION

```
#include <iostream>
using namespace std;

int main() {
    const int N = 3;
    int m[N][N];
    cout << "Enter 9 numbers:" << endl;
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            cin >> m[i][j];

    bool identity = true;
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) {
            if (i == j && m[i][j] != 1) identity = false;   // diagonal must be 1
            if (i != j && m[i][j] != 0) identity = false;   // rest must be 0
        }

    if (identity) cout << "It is an identity matrix" << endl;
    else          cout << "It is not an identity matrix" << endl;
    return 0;
}
```

SAMPLE INPUT

```
1 0 0 0 1 0 0 0 1
```

OUTPUT

```
Enter 9 numbers:
It is an identity matrix
```

Program 16: Add and multiply two matrices

Handout reference: L12-L13

Addition is element by element. Multiplication takes row-times-column sums (three nested loops).

SOLUTION

```
#include <iostream>
#include <iomanip>
using namespace std;

const int N = 3;

void printMatrix(const char *title, int m[][N]) {
    cout << title << endl;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) cout << setw(5) << m[i][j];
        cout << endl;
    }
}

int main() {
    int A[N][N] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    int B[N][N] = {{9, 8, 7}, {6, 5, 4}, {3, 2, 1}};
    int sum[N][N], prod[N][N];

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) {
            sum[i][j] = A[i][j] + B[i][j];
            prod[i][j] = 0;
            for (int k = 0; k < N; k++)    // row i of A times column j of B
                prod[i][j] += A[i][k] * B[k][j];
        }

    printMatrix("A + B =", sum);
    printMatrix("A x B =", prod);
    return 0;
}
```

OUTPUT

```
A + B =
   10   10   10
   10   10   10
   10   10   10
A x B =
   30   24   18
   84   69   54
  138  114   90
```

Program 17: Sum of both diagonals

Handout reference: L12-L13

Main diagonal: [i][i]. Secondary diagonal: [i][N-1-i]. One loop handles both.

SOLUTION

```
#include <iostream>
using namespace std;

int main() {
    const int N = 3;
    int m[N][N] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 10}};

    int main_d = 0, second_d = 0;
    for (int i = 0; i < N; i++) {
        main_d   += m[i][i];              // top-left to bottom-right
        second_d += m[i][N - 1 - i];      // top-right to bottom-left
    }
    cout << "Main diagonal sum = " << main_d << endl;
    cout << "Secondary diagonal sum = " << second_d << endl;
    return 0;
}
```

OUTPUT

```
Main diagonal sum = 16
Secondary diagonal sum = 15
```

Level 4: Strings

Program 18: Write your own strlen, strcpy, strcmp, strcat

Handout reference: L12, L17 (handout exercise)

A C-string is a char array ending with '\0'. Every function walks until it meets that marker.

SOLUTION

```
#include <iostream>
using namespace std;

int myStrlen(const char *s) {
    int n = 0;
    while (s[n] != '\0') n++;
    return n;
}

void myStrcpy(char *dest, const char *src) {
    int i = 0;
    while ((dest[i] = src[i]) != '\0') i++;
}

int myStrcmp(const char *a, const char *b) {
    int i = 0;
    while (a[i] != '\0' && a[i] == b[i]) i++;
    return a[i] - b[i];                    // 0 means equal
}

void myStrcat(char *dest, const char *src) {
    int n = myStrlen(dest), i = 0;
    while ((dest[n + i] = src[i]) != '\0') i++;
}

int main() {
    char s1[50] = "Hello", s2[50];
    cout << "Length of s1 = " << myStrlen(s1) << endl;

    myStrcpy(s2, s1);
    cout << "Copy of s1 = " << s2 << endl;

    int r = myStrcmp("apple", "banana");
    cout << "apple vs banana: " << (r < 0 ? "apple comes first" : "banana comes first") << endl;
    cout << "cat vs cat = " << myStrcmp("cat", "cat") << endl;

    myStrcat(s1, " World");
    cout << "After concatenation = " << s1 << endl;
    return 0;
}
```

OUTPUT

```
Length of s1 = 5
Copy of s1 = Hello
apple vs banana: apple comes first
cat vs cat = 0
After concatenation = Hello World
```

Program 19: Reverse a string and check palindrome

Handout reference: L12

Build the reversed copy, then compare it with the original character by character.

SOLUTION

```
#include <iostream>
using namespace std;

int main() {
    char str[100], rev[100];
    cout << "Enter a word:" << endl;
    cin >> str;

    int len = 0;
    while (str[len] != '\0') len++;

    for (int i = 0; i < len; i++) rev[i] = str[len - 1 - i];
    rev[len] = '\0';                        // always close the string
    cout << "Reversed: " << rev << endl;

    bool palindrome = true;
    for (int i = 0; i < len; i++)
        if (str[i] != rev[i]) palindrome = false;

    cout << str << (palindrome ? " is a palindrome" : " is not a palindrome") << endl;
    return 0;
}
```

SAMPLE INPUT

```
level
```

OUTPUT

```
Enter a word:
Reversed: level
level is a palindrome
```

Program 20: Count vowels, consonants and words

Handout reference: L12

Check each character: letters are vowels or consonants, and a new word starts after a space.

SOLUTION

```
#include <iostream>
#include <cctype>
using namespace std;

int main() {
    char line[200];
    cout << "Enter a sentence:" << endl;
    cin.getline(line, 200);

    int vowels = 0, consonants = 0, words = 0;
    bool inWord = false;

    for (int i = 0; line[i] != '\0'; i++) {
        char c = tolower(line[i]);
        if (isalpha(c)) {
            if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u') vowels++;
            else consonants++;
        }
        if (c != ' ' && !inWord) { words++; inWord = true; }   // a new word begins
        else if (c == ' ')       inWord = false;
    }
    cout << "Vowels = " << vowels << endl;
    cout << "Consonants = " << consonants << endl;
    cout << "Words = " << words << endl;
    return 0;
}
```

SAMPLE INPUT

```
Learning C plus plus is fun
```

OUTPUT

```
Enter a sentence:
Vowels = 7
Consonants = 15
Words = 6
```

Program 21: Check whether two words are anagrams

Handout reference: L12

Count each letter of the first word up and of the second word down. All zeros means anagram.

SOLUTION

```
#include <iostream>
#include <cctype>
using namespace std;

int main() {
    char a[50], b[50];
    cout << "Enter two words:" << endl;
    cin >> a >> b;

    int count[26] = {0};
    for (int i = 0; a[i] != '\0'; i++) count[tolower(a[i]) - 'a']++;
    for (int i = 0; b[i] != '\0'; i++) count[tolower(b[i]) - 'a']--;

    bool anagram = true;
    for (int i = 0; i < 26; i++)
        if (count[i] != 0) anagram = false;

    if (anagram) cout << a << " and " << b << " are anagrams" << endl;
    else         cout << a << " and " << b << " are not anagrams" << endl;
    return 0;
}
```

SAMPLE INPUT

```
listen silent
```

OUTPUT

```
Enter two words:
listen and silent are anagrams
```

Program 22: Print the ASCII table

Handout reference: L17 (handout exercise)

A char is just a small number. Cast the number to char to see the symbol.

SOLUTION

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    cout << "Printable ASCII characters (32 to 126):" << endl;
    for (int code = 32; code <= 126; code++) {
        cout << setw(3) << code << " = " << char(code) << "    ";
        if ((code - 31) % 6 == 0) cout << endl;     // 6 entries per line
    }
    cout << endl;
    return 0;
}
```

OUTPUT

```
Printable ASCII characters (32 to 126):
 32 =       33 = !     34 = "     35 = #     36 = $     37 = %
 38 = &     39 = '     40 = (     41 = )     42 = *     43 = +
 44 = ,     45 = -     46 = .     47 = /     48 = 0     49 = 1
 50 = 2     51 = 3     52 = 4     53 = 5     54 = 6     55 = 7
 56 = 8     57 = 9     58 = :     59 = ;     60 = <     61 = =
 62 = >     63 = ?     64 = @     65 = A     66 = B     67 = C
 68 = D     69 = E     70 = F     71 = G     72 = H     73 = I
 74 = J     75 = K     76 = L     77 = M     78 = N     79 = O
 80 = P     81 = Q     82 = R     83 = S     84 = T     85 = U
 86 = V     87 = W     88 = X     89 = Y     90 = Z     91 = [
 92 = \     93 = ]     94 = ^     95 = _     96 = `     97 = a
 98 = b     99 = c    100 = d    101 = e    102 = f    103 = g
104 = h    105 = i    106 = j    107 = k    108 = l    109 = m
110 = n    111 = o    112 = p    113 = q    114 = r    115 = s
116 = t    117 = u    118 = v    119 = w    120 = x    121 = y
122 = z    123 = {    124 = |    125 = }    126 = ~
```

Level 5: Pointers

Program 23: Swap two numbers using pointers

Handout reference: L14

Pass addresses with &, then use * inside the function to change the original variables.

SOLUTION

```
#include <iostream>
using namespace std;

void swapNumbers(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 10, y = 25;
    cout << "Before: x = " << x << ", y = " << y << endl;
    swapNumbers(&x, &y);
    cout << "After:  x = " << x << ", y = " << y << endl;
    return 0;
}
```

OUTPUT

```
Before: x = 10, y = 25
After:  x = 25, y = 10
```

Program 24: Traverse an array using pointer arithmetic

Handout reference: L15

An array name is the address of its first element. p + i moves i elements forward.

SOLUTION

```
#include <iostream>
using namespace std;

int main() {
    int a[5] = {10, 20, 30, 40, 50};
    int *p = a;                          // p points to a[0]

    cout << "Using *(p + i):" << endl;
    for (int i = 0; i < 5; i++)
        cout << "a[" << i << "] = " << *(p + i) << endl;

    int sum = 0;
    for (int *q = a; q < a + 5; q++)     // move the pointer itself
        sum += *q;
    cout << "Sum = " << sum << endl;
    return 0;
}
```

OUTPUT

```
Using *(p + i):
a[0] = 10
a[1] = 20
a[2] = 30
a[3] = 40
a[4] = 50
Sum = 150
```

Program 25: Bubble sort using pointers

Handout reference: L14 (handout sample)

The same sort as before, but every access uses *(arr + j) instead of arr[j].

SOLUTION

```
#include <iostream>
using namespace std;

void bubbleSort(int *arr, int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (*(arr + j) > *(arr + j + 1)) {
                int temp = *(arr + j);
                *(arr + j) = *(arr + j + 1);
                *(arr + j + 1) = temp;
            }
}

int main() {
    int a[6] = {64, 25, 12, 22, 11, 90};
    bubbleSort(a, 6);                    // array name decays to a pointer

    cout << "Sorted: ";
    for (int i = 0; i < 6; i++) cout << a[i] << " ";
    cout << endl;
    return 0;
}
```

OUTPUT

```
Sorted: 11 12 22 25 64 90
```

Program 26: Fill, print and add an array through pointers

Handout reference: L14-L15

Pass the array as a pointer plus its size. Each function then works on the original data.

SOLUTION

```
#include <iostream>
using namespace std;

void fillArray(int *arr, int n) {
    for (int i = 0; i < n; i++) cin >> *(arr + i);
}

void printArray(const int *arr, int n) {
    for (int i = 0; i < n; i++) cout << *(arr + i) << " ";
    cout << endl;
}

int arraySum(const int *arr, int n) {
    int sum = 0;
    for (int i = 0; i < n; i++) sum += *(arr + i);
    return sum;
}

int main() {
    int a[5];
    cout << "Enter 5 numbers:" << endl;
    fillArray(a, 5);

    cout << "You entered: ";
    printArray(a, 5);
    cout << "Sum = " << arraySum(a, 5) << endl;
    cout << "Average = " << arraySum(a, 5) / 5.0 << endl;
    return 0;
}
```

SAMPLE INPUT

```
10 20 30 40 55
```

OUTPUT

```
Enter 5 numbers:
You entered: 10 20 30 40 55
Sum = 155
Average = 31
```

Level 6: Structures and Files

Program 27: Student records: array of structures, find the topper

Handout reference: L20

An array of structures stores many records. Loop through it to compare and total the marks.

SOLUTION

```
#include <iostream>
#include <iomanip>
using namespace std;

struct Student {
    char name[20];
    int marks;
};

int main() {
    Student s[4] = {{"Ali", 78}, {"Sara", 92}, {"Bilal", 65}, {"Hina", 88}};

    int top = 0;
    double total = 0;
    cout << left << setw(10) << "Name" << "Marks" << endl;
    for (int i = 0; i < 4; i++) {
        cout << left << setw(10) << s[i].name << s[i].marks << endl;
        total += s[i].marks;
        if (s[i].marks > s[top].marks) top = i;     // remember the best index
    }

    cout << "Topper: " << s[top].name << " (" << s[top].marks << ")" << endl;
    cout << "Class average: " << total / 4 << endl;
    return 0;
}
```

OUTPUT

```
Name      Marks
Ali       78
Sara      92
Bilal     65
Hina      88
Topper: Sara (92)
Class average: 80.75
```

Program 28: Write text to a file, then read it back and count lines

Handout reference: L18

ofstream writes to a file. ifstream plus getline reads it back one line at a time.

SOLUTION

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    ofstream out("notes.txt");             // create and open for writing
    out << "Line one\n";
    out << "Line two\n";
    out << "Line three\n";
    out.close();

    ifstream in("notes.txt");              // open the same file for reading
    string line;
    int count = 0;
    while (getline(in, line)) {
        count++;
        cout << count << ": " << line << endl;
    }
    in.close();

    cout << "Total lines = " << count << endl;
    return 0;
}
```

OUTPUT

```
1: Line one
2: Line two
3: Line three
Total lines = 3
```